

Automatiser une série d'analyses en R

De $Y \sim X$ à $Y \sim$ plusieurs X

SeRious

Enjeu

Dans les étapes précédentes, on a appris à construire un prompt à partir d'une sortie statistique.

On a aussi vu qu'un objet R peut être inspecté, capturé, transformé en texte, puis réutilisé.

Cette fiche introduit une autre forme d'automatisation : **répéter une même analyse pour plusieurs variables explicatives**.

L'idée n'est pas de multiplier les modèles sans réfléchir.

L'objectif est de rendre visible une logique très fréquente dans un workflow statistique :

```
une analyse isolée
→ une fonction
→ une application répétée
→ une liste de résultats
→ des sorties réutilisables
```

Situation de départ

On dispose d'un questionnaire contenant une variable à expliquer, par exemple :

```
intention_achat
```

et plusieurs variables explicatives possibles :

```
prix_percu
plaisir
naturalite
confiance
ancrage_local
usage_numerique
sensibilite_env
```

On pourrait ajuster les modèles un par un :

```
intention_achat ~ prix_percu
intention_achat ~ plaisir
intention_achat ~ naturalite
```

Mais cette stratégie devient vite répétitive, fragile et difficile à maintenir.

Pourquoi automatiser ?

Automatiser ne signifie pas abandonner le raisonnement statistique.

Au contraire, cela oblige à préciser ce qui reste fixe et ce qui varie.

Dans cette situation :

Élément	Rôle dans le workflow
<code>intention_achat</code>	variable à expliquer, donc partie fixe
<code>variables_x</code>	variables explicatives, donc partie variable
<code>FactoMineR::LinearModel()</code>	méthode appliquée à chaque formule
<code>modeles_univaries</code>	objet qui stockera tous les modèles
<code>textes_modeles_univaries</code>	objet qui stockera les sorties textuelles

Le passage important est donc :

```
Y ~ X
→ Y ~ X1
→ Y ~ X2
→ Y ~ X3
→ liste de modèles
→ liste de textes
```

Définir ce qui reste fixe

On commence par stocker le nom de la variable à expliquer dans un objet.

```
y <- "intention_achat"
```

Cette écriture est importante : on ne code plus directement le nom de la variable dans chaque formule. On le place dans un objet qui pourra être réutilisé.

Définir ce qui varie

Les variables explicatives sont stockées dans un vecteur.

```
variables_x <- c(
  "prix_percu",
  "plaisir",
  "naturalite",
  "confiance",
  "ancrage_local",
  "usage_numerique",
  "sensibilite_env"
)
```

Ce vecteur joue le rôle d'une liste de tâches : chaque élément correspond à un modèle à ajuster.

Construire une première formule

Avant d'écrire une boucle ou une fonction, il faut vérifier que la logique fonctionne sur un cas simple.

On prend la première variable explicative :

```
premier_x <- variables_x[1]
```

Puis on construit une formule sous forme de texte :

```
formule_texte <- paste(y, "~", premier_x)

formule_texte
```

Si `y` vaut `"intention_achat"` et `premier_x` vaut `"prix_percu"`, on obtient :

```
intention_achat ~ prix_percu
```

Ce n'est pas encore une formule R : c'est une chaîne de caractères.

Transformer le texte en formule R

Les fonctions de modélisation attendent généralement une formule, pas seulement du texte. On transforme donc la chaîne de caractères avec `as.formula()`.

```
formule_exemple <- as.formula(formule_texte)

formule_exemple
```

La transformation est essentielle :

```
"intention_achat ~ prix_percu"
→ intention_achat ~ prix_percu
```

La première forme est du texte.

La seconde est un objet formule utilisable par une fonction de modélisation.

Ajuster un premier modèle

On peut maintenant ajuster un premier modèle.

```
modele_exemple <- FactoMineR::LinearModel(
  formule_exemple,
  data = questionnaire,
  selection = "none"
)
```

Cette étape sert de test.

Avant d'automatiser, on vérifie que le cas simple fonctionne.

Lire les sorties principales

L'objet retourné contient plusieurs sorties.

On peut par exemple inspecter :

```
modele_exemple$Ftest
modele_exemple$Ttest
```

Ces deux sorties correspondent à deux niveaux de lecture.

Sortie	Rôle
Ftest	tester l'effet global de la variable explicative
Ttest	examiner les coefficients du modèle

Cette distinction restera valable pour les autres modèles ajustés ensuite.

Écrire une fonction

Lorsque les mêmes instructions doivent être répétées, on peut les enfermer dans une fonction.

```
ajuster_un_modele <- function(x) {

  formule_texte <- paste(y, "~", x)
  formule <- as.formula(formule_texte)

  modele <- FactoMineR::LinearModel(
    formule,
    data = questionnaire,
    selection = "none"
  )
}
```

```
)

modele
}
```

Cette fonction reçoit le nom d'une variable explicative, construit la formule correspondante, ajuste le modèle, puis retourne le résultat.

Par exemple :

```
ajuster_un_modele("prix_percu")
```

revient à ajuster :

```
intention_achat ~ prix_percu
```

Ce que généralise la fonction

La fonction généralise la structure suivante :

```
prendre une variable X
→ construire la formule Y ~ X
→ ajuster LinearModel()
→ retourner le modèle
```

Le code devient plus court, mais surtout plus systématique.
On réduit ainsi les erreurs de copier-coller.

Appliquer la fonction à toutes les variables

On applique maintenant la fonction à chaque variable explicative avec `lapply()`.

```
modeles_univaries <- lapply(
  variables_x,
  ajuster_un_modele
)
```

La fonction `lapply()` prend chaque élément de `variables_x` et lui applique la fonction `ajuster_un_modele()`.

Autrement dit, R exécute automatiquement :

```
ajuster_un_modele("prix_percu")
ajuster_un_modele("plaisir")
ajuster_un_modele("naturalite")
...
```

Le résultat est une liste de modèles.

Nommer les éléments de la liste

Une liste sans noms est difficile à relire.

On ajoute donc les noms des variables explicatives.

```
names(modeles_univaries) <- variables_x
```

On peut ensuite retrouver un modèle précis avec :

```
modeles_univaries[["prix_percu"]]
```

Cet objet correspond au modèle :

```
intention_achat ~ prix_percu
```

Inspecter les modèles produits

On peut vérifier le nombre de modèles obtenus et leurs noms.

```
length(modeles_univaries)
names(modeles_univaries)
```

On peut aussi examiner les sorties des deux premiers modèles :

```
modeles_univaries[[1]]$Ftest
modeles_univaries[[1]]$Ttest

modeles_univaries[[2]]$Ftest
modeles_univaries[[2]]$Ttest
```

L'objectif n'est pas d'afficher toute la liste dans la console.

Il est préférable de vérifier quelques éléments, puis de garder l'objet complet pour la suite.

Objet créé : `modeles_univaries`

L'objet `modeles_univaries` est une liste.

Chaque élément de cette liste est un résultat `LinearModel()`.

Élément	Modèle ajusté
prix_percu	intention_achat ~ prix_percu
plaisir	intention_achat ~ plaisir
naturalite	intention_achat ~ naturalite
confiance	intention_achat ~ confiance

Cette organisation permet de conserver tous les résultats sans encombrer la console.

Capturer chaque modèle sous forme de texte

On retrouve maintenant une logique vue dans les fiches précédentes : capturer une sortie R pour la rendre réutilisable.

```
textes_modeles_univaries <- lapply(
  modeles_univaries,
  function(modele) {
    paste(
      capture.output(print(modele)),
      collapse = "\n"
    )
  }
)
```

Cette instruction transforme chaque modèle en texte.

Rôle de `capture.output()`

`capture.output()` récupère ce qui aurait été imprimé dans la console.

```
capture.output(print(modele))
```

La sortie capturée peut ensuite être stockée, relue, comparée ou injectée dans un prompt.

Rôle de `paste(..., collapse = "\n")`

`capture.output()` renvoie généralement plusieurs lignes.

La fonction `paste(..., collapse = "\n")` rassemble ces lignes dans une seule chaîne de caractères, tout en conservant les retours à la ligne.

```
paste(
  capture.output(print(modele)),
  collapse = "\n"
)
```

Le résultat est un texte structuré.

Objet créé : `textes_modeles_univaries`

L'objet `textes_modeles_univaries` est une liste de textes.

Chaque élément contient la sortie complète d'un modèle.

Élément	Contenu
<code>prix_percu</code>	sortie texte du modèle <code>intention_achat ~ prix_percu</code>
<code>plaisir</code>	sortie texte du modèle <code>intention_achat ~ plaisir</code>
<code>naturalite</code>	sortie texte du modèle <code>intention_achat ~ naturalite</code>

On peut examiner un premier texte avec :

```
premier_texte <- textes_modeles_univaries[[1]]

substr(premier_texte, 1, 1200)
nchar(premier_texte)
```

Ce que cette case ne fait pas

Cette automatisation ne dit pas quelle variable est la plus importante, ni quel modèle doit être retenu. Elle produit une série de modèles comparables et réutilisables.

Il faut donc rester prudent :

- plusieurs modèles univariés ne remplacent pas un modèle multivarié ;
- répéter des tests augmente le risque de surinterprétation ;
- une association statistique n'est pas une preuve de causalité ;
- les résultats doivent être lus en fonction de la question posée.

L'automatisation organise le travail, mais elle ne remplace pas le jugement statistique.

Lien avec les prompts

Les textes capturés peuvent ensuite être utilisés pour construire des prompts.

Par exemple, on pourrait écrire un prompt demandant :

```
Voici plusieurs modèles univariés reliant intention_achat à différentes variables explicatives.
Compare les résultats sans inventer de valeurs absentes.
Distingue les effets globaux, les coefficients et les limites d'interprétation.
```

On retrouve donc le fil conducteur :

```
objet statistique
→ sortie capturée
→ texte contrôlé
→ prompt inspectable
```

Lien avec `condes()`

La case suivante introduit `condes()`.

Le lien doit être compris avec prudence : `condes()` n'est pas simplement une boucle de modèles `LinearModel()`. Mais cette case rend visible une idée que `condes()` généralise : **décrire systématiquement une variable quantitative à partir des autres variables disponibles**.

En particulier, `condes()` adapte la logique descriptive selon la nature des variables :

Type de relation	Exemple de logique
quantitative quantitative	corrélations
quantitative qualitative	comparaisons de moyennes, rapports de corrélation, tests associés

La boucle `Y ~ plusieurs X` prépare donc l'esprit à une description plus automatique et plus complète.

Objets disponibles à la fin

À la fin du workflow, plusieurs objets sont disponibles.

Objet	Rôle
<code>y</code>	nom de la variable à expliquer
<code>variables_x</code>	noms des variables explicatives testées
<code>formule_exemple</code>	première formule construite automatiquement
<code>modele_exemple</code>	premier modèle ajusté sans boucle
<code>ajuster_un_modele()</code>	fonction qui ajuste un modèle pour une variable X
<code>modeles_univaries</code>	liste des modèles ajustés
<code>textes_modeles_univaries</code>	liste des sorties transformées en texte

Ces objets peuvent être inspectés, sauvegardés ou transmis à une étape ultérieure.

À retenir

Cette fiche montre comment passer d'une analyse isolée à une logique systématique.

Le message clé est :

```
si une opération est répétée plusieurs fois,  
elle peut devenir une fonction,  
puis être appliquée automatiquement.
```

L'automatisation ne supprime pas la compréhension.

Elle oblige au contraire à clarifier :

- ce qui reste fixe ;
- ce qui varie ;
- ce qui doit être stocké ;
- ce qui doit être affiché ;
- ce qui pourra être réutilisé.

Synthèse

```
Y ~ X  
→ Y ~ plusieurs X  
→ fonction  
→ lapply()
```

- liste de modèles
- liste de sorties textuelles
- matériau pour prompts ou descriptions automatiques

Cette logique prépare la suite du parcours : comprendre pourquoi des fonctions descriptives comme `condes()` et, plus loin, des packages comme NaileR peuvent systématiser la production d'artefacts interprétables.